TOKCIM: A 65nm All-Digital Compute-in-Memory Transformer Accelerator with Advanced Weight Sparsity Handling and SoftMax Acceleration

Shaoxuan LI[1,2], Zikun WEI[1], Ruihao HE[1], Cong WANG[1], Ye GU[1], Tianchu DONG[1], Yucheng CAI[1] and Hongwu JIANG[1]

1. Microelectronics Thrust, Function Hub, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China
2. Guangdong-Macao Joint Laboratory for Modular Chip Design and Testing, University of Macau, Macau, China
hongwujiang@hkust-gz.edu.cn

2026 IEEE Custom Integrated Circuits Conference (CICC) | 979-8-3315-7210-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/CICC65509.2026.11509500

***Abstract*—This paper presents TOKCIM, a Transformer accelerator based on digital compute-in-memory (DCIM) structure. TOKCIM adopts all-zero initialization (AZI) characteristics in the DCIM macros, together with group pruning of weights to fully leverage the benefits of AZI, speeding up both weight loading process and in-memory computing process. The SoftMax unit (SMU) in TOKCIM employs an optimal value selection strategy. By selecting several optimal values among inputs and performing SoftMax operations only on the selected values, the proposed SMU can significantly reduce the overhead of activation calculations with negligible degradation on performance. Fabricated in 65-nm CMOS process and benchmarked on Q8BERT model and GLUE dataset, TOKCIM achieves a measured energy efficiency of 5.73 TOPS/W and a compute efficiency of 0.29 TOPS/mm², demonstrating state-of-the-art performance.**

***Keywords*—Transformer, compute-in-memory, SoftMax, accelerator**

I. Introduction

Transformers constitute the core of many emerging artificial intelligence (AI) applications, particularly in the fields of natural language processing and computer vision. Nevertheless, they face considerable challenges stemming from their continuously expanding model sizes and the extensive use of multiply-accumulate (MAC) operations. These factors contribute to frequent data transfers and increased computational costs. In response, a trend toward digital compute-in-memory (DCIM) has emerged in the design of accelerators tailored for Transformers. This approach shows promise by enabling matrix multiplication to be executed within memory macros, which mitigates data movement overhead while preserving computational accuracy. To leverage the potential of DCIM in accelerating sparse MAC operations, several recent studies [1]-[2] have incorporated sparsity-handling strategies to reduce computational overhead. However, these works seldom explore leveraging sparsity to enhance energy efficiency during the process of loading weights into the DCIM macro, which is an increasingly important issue since weight loading becomes more frequent as the model size continuously increases. Another challenge is the Transformer's dependence on the SoftMax function, which contains abundant hardware-unfriendly non-linear computations.

The proposed design, TOKCIM, addresses the aforementioned challenges through three key contributions as shown in Fig. 1. First, an all-zero initialization (AZI) feature is integrated in the DCIM macro, which synergistically combined with group weight pruning to fully leverage the benefits of AZI for enhanced performance. Second, the SoftMax unit (SMU) employs an optimal value selection strategy based on the fact that SoftMax outputs are dominated by a limited number of large input values. By computing only on this critical subset, the SMU avoids unnecessary calculations with negligible impact on result accuracy. Third, to overcome the lack of programmability and low instruction decoding efficiency for both DCIM and the specific SoftMax operations, we propose dedicated concise instructions that accelerate the fetching and decoding processes, particularly for the optimal value selection.

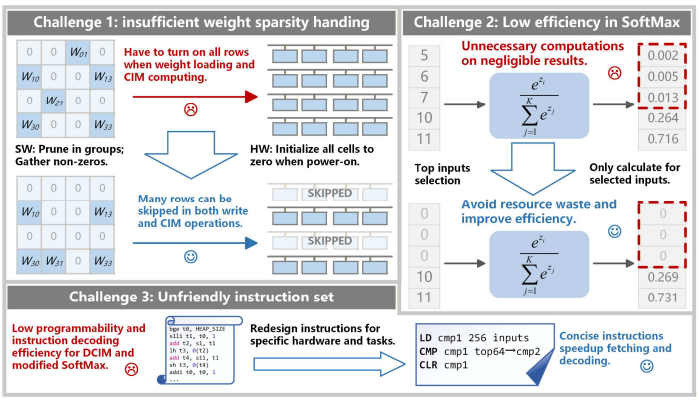


Fig. 1. Challenges and solutions for DCIM-based Transformer accelerators.

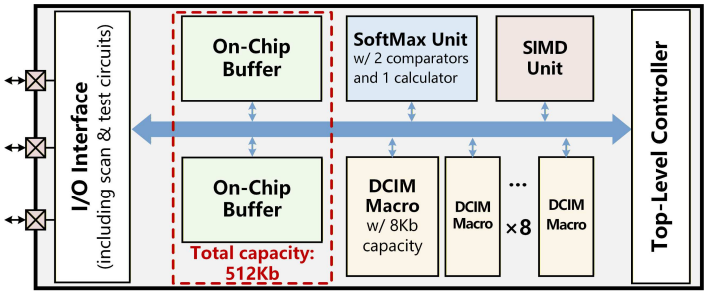


Fig. 2. Overall architecture of TOKCIM.

II. Overall Architecture

Fig. 2 illustrates the overall architecture of TOKCIM, which comprises 8 DCIM macros, an SMU, an SIMD unit, multiple on-chip buffers, I/O interface circuits, and a top-level controller. The primary innovations lie in the DCIM macros and the SMU. The DCIM macros are mainly responsible for MAC computations, leveraging weight sparsity to improve computational speed and energy efficiency. The

This work was supported in part by the National Natural Science Foundation of China (Grant No. 62304195), the Guangdong Science and Technology Department (No.2025B1212150003), and GDIC.

979-8-3315-7210-5/26/$31.00 ©2026 IEEE
Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:16 UTC from IEEE Xplore. Restrictions apply.

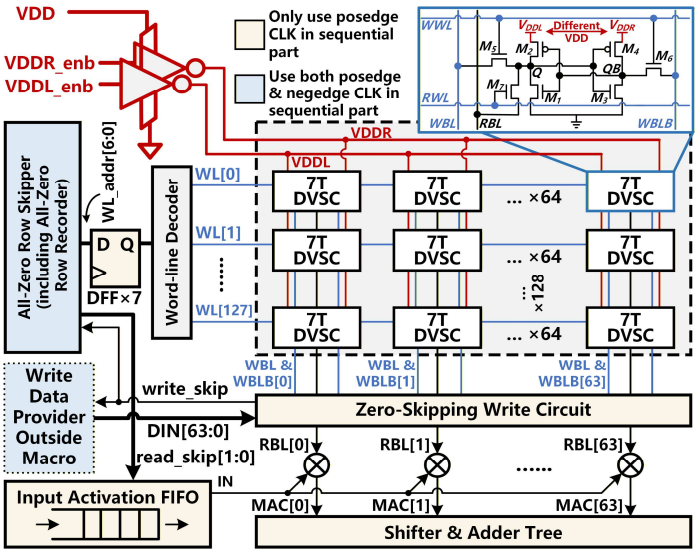


Fig. 3. Overall structure of DCIM macro with the schematic of DVSC.

SMU implements the optimal value selection strategy using a dual-comparator ping-pong structure to support larger input depth, while a dedicated calculator performs the SoftMax function exclusively on the selected inputs. A shared 512Kb buffer provides flexible storage, dynamically serving for input activations, intermediate results, and output activations. The I/O interface supports two operational modes: acting as a co-processor to a host processing unit, or collaborating with another TOKCIM chip to enable the deployment of large Transformer models. All components are interconnected via a shared data bus.

III. DCIM Macro with All-Zero Initialization

The schematic of 7T decoupled-VDD SRAM cell (DVSC) is presented inside Fig. 3. Its key feature is the use of separate power supplies for the left and right inverters (VDDL and VDDR, respectively). Unlike a standard 6T SRAM cell with an uncertain power-on state, the initial value of the 7T DVSC can be deterministically set by controlling the power-on sequence of VDDL and VDDR. For instance, powering on VDDR before VDDL activates the right-side inverter first with M4 on and M3 off, forcing the cell to a state of Q = 0 and QB = 1. This mechanism enables a one-step, all-zero initialization (AZI) across the entire memory array.

The overall structure of proposed DCIM macro is composed of 8192 DVSCs organized in a 128 (rows) × 64 (columns) array. To leverage the AZI for efficiency, the macro incorporates a zero-skipping write circuit (ZSWC) and an all-zero row skipper (AZRS), enabling row-wise skipping during both write and DCIM operations. The AZRS and the external write data provider can work at double data rate, updating the data at both positive and negative clock edges. An internal all-zero row recorder (AZRR) within the AZRS flags whether a row is entire zero during write operations. If a row is intended to be all-zero, the AZI renders the actual write operation unnecessary, allowing it to be skipped. The choice of 64 columns per row represents a deliberate design trade-off. A smaller number of columns increases the probability of a row being all-zero, maximizing the benefits of AZI and skipping. However, an excessively low column count for a given memory capacity would lead to an unreasonably high number of rows, increasing bit-line parasitic capacitance and adversely impacting energy efficiency and critical delay. The impact of this granularity choice is analyzed in Chapter VI through various pruning strategies.

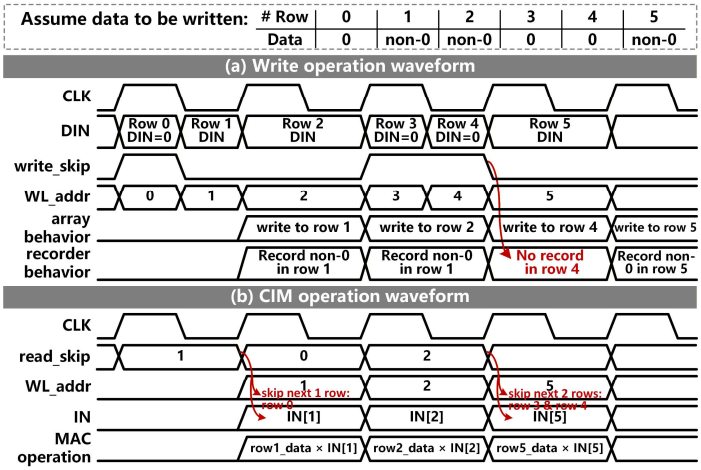


Fig. 4. Waveform of DCIM macro: (a) Write operation; (b) CIM operation.

Write operation waveform is illustrated in Fig. 4(a). The ZSWC first checks the data input (DIN) in the first half-cycle. If it is all-zero, the circuit asserts *write_skip* as high to request the next DIN, while the AZRS increments the wordline address and AZRR's internal pointer. As shown in Fig. 4(b), during CIM operations, AZRS utilizes the recorded information from AZRR to skip all-zero rows by setting a *read_skip* signal. This signal directs the input activation FIFO to skip the corresponding number of input bits. The *read_skip* signal is 2-bits wide, allowing it to skip up to three consecutive all-zero rows, a number chosen based on the statistically low probability of four or more consecutive all-zero rows.

IV. Top-k Selector for SoftMax Acceleration

Fig. 5 illustrates the proposed SoftMax unit (SMU) design based on the optimal value selection strategy. By identifying the top-*k* input values and performing SoftMax operations only on the selected subset, the overhead of hardware-unfriendly computations such as exponential computations and divisions can be significantly reduced, while introducing only negligible errors. The core module for optimal value selection is a top-*k* selector, implemented with two comparators in a ping-pong structure. A single comparator supports up to 256-to-192 selection for 16-bit data. Detailed in Fig. 5(a), the top-k selection operation, demonstrated via an 8-to-4 selection example, relies on a shift buffer (which stores all inputs) and three key signals:

- *counter1*: Tracks the cumulative count of data identified as potential top-*k* values (i.e., the number of previously output data + the number of data that MSB is 1 in the current cycle.
- *counter2*: Indicates the number of bit shifted. It increments with each left shift of the buffer, starting from 0.
- *MSB_all0*: indicate whether MSBs of all data in shift buffer are '0'. If the MSBs of all data are '0', this flag is set to '1'; otherwise, it is '0'.

The operational logic for the shift buffer is as follows: if *counter1* ≤ *k* and *MSB_all0* = 0 (case 1), all data with MSB =1 in the shift buffer are selected as temporary outputs (*out_temp*), and their positions in the buffer are reset; if *counter1* < *k* and *MSB_all0* = 1 (case 2), no conclusive selection can be made by comparing MSBs. The entire buffer is left-shifted by one bit.; if *counter1* = *k* and *MSB_all0* = 1 (case 3), The selection process is completed; if *counter1* > *k* (case 4), all data with MSB = 0 are discarded (reset) while the remaining data requires further comparison, prompting a left shift.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:16 UTC from IEEE Xplore. Restrictions apply.

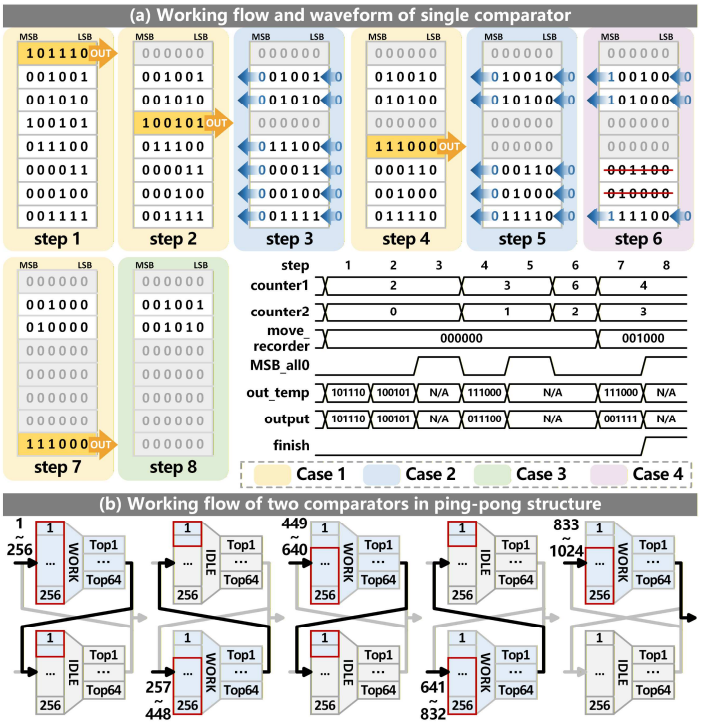


Fig. 5. (a) Working flow and waveform of single comparator; (b) Working flow of top-*k* selector consisting of two comparators in ping-pong structure.

To recover the original data, a 16-bit register (*move_recorder*) tracks the reason for each left shift. A shift due to *case 2* leaves the corresponding bit as '0', while a shift from *case 4* set it to be '1'. The final output value *output* is then reconstructed from temporary output value *out_temp* in equation (1):

$$output = (out_temp << counter2) \mid move_recorder \quad (1)$$

In practice, a single comparator's 256-input limit is insufficient for large Transformer models. However, scaling the input depth directly may cause unacceptable propagation delays of *counter1* and *MSB_all0*, since they need to go through all the MSBs in shift buffer. As a countermeasure, SMU employs two top-*k* comparators in a ping-pong structure, enabling the handling of complex tasks with large input depths. An example of 1024-to-64 selection is demonstrated in Fig. 5(b). The first 256 input values are processed by Comparator 1, and its top-64 results are passed to Comparator 2. Subsequently, the following 192 inputs are transmitted into the comparator that is currently holding 64 values, and the new top-64 results are passed to the other comparator or the downstream SoftMax calculation modules. This pipelined operation allows for continuous processing of extensive input sequences.

V. Instruction Set Extensions

For the accelerators customized for specific tasks like TOKCIM, supporting CISC-like instruction set can bring lots of benefits, because CISC-like instructions are good at describing a specific task with a smaller number of instructions at the expense of the diversity of the tasks able to be described. The feature of specifically optimizing for particular tasks precisely aligns with the thinking behind hardware design. Fig. 6 illustrates the TOKCIM's customized high-level CISC-like instruction sets in the function of CIM and SoftMax. While the compatibility of low-level, RISC-like instructions including various

(a) Summary of proposed instructions

Type	Inst.	Description	Keywords
CIM	AZI	Execute all-zero initialize operation in DCIM macros.	**Select hardware** (# of macro)
	WT	Write data to DCIM macros.	**Src.** (addr. of global buf.) **Dest.** (# of macro, # of row) **Iterations** (# of rows to be written)
	RD	Directly readout the data from DCIM macros.	**Select hardware** (# of macro) **Initial row** (from 0 to 127) **# of rows to be read** **Dest.** (addr. of global buf.)
	CIM	Execute compute-in-memory operation in DCIM macros.	**Select hardware** (# of macro) **Input precision** (4 / 8 / 16) **Input src.** (addr. of global buf.) **Weight precision** (4 / 8 / 16) **Iterations** (depend on matrix size)
Soft-Max	LD	Load data to one comparator or the calculator.	**Src.** (addr. of global buf.) **Dest.** (comp1 / 2 / calc, row in buf.) **Data format** (INT / FP / BF) **# of inputs** (64, 128, 192, 256, ...)
	CMP	Invoke one comparator for top value selection.	**Select hardware** (comp1 / 2) **Top-k** (16, 32, 64, 128, ...) **Dest.** (comp1 / 2 / calc / out)
	SM	Invoke the calculator to execute SoftMax calculation.	**Data format** (INT / FP / BF) **# of inputs** (64, 128, 192, 256, ...) **Dest.** (addr. of global buf.)
	CLR	Clear the data in comparators or calculator.	**Select hardware** (comp1 / 2 / calc)

(b) Complexity comparison in describing the task "1024→64"

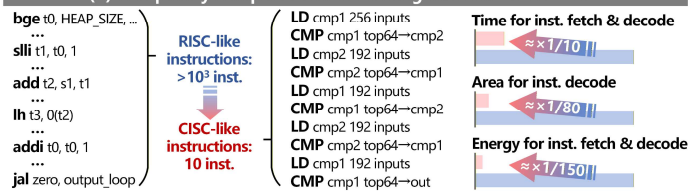


Fig. 6. (a) Proposed instructions for CIM and SoftMax operations; (b) Complexity comparison of different instruction sets for describing the example task.

vector operations such as add/subtract/multiply is also preserved, and these instructions can be directly executed in the SIMD unit. The top-level controller is in charge of identifying the instructions for different circuit modules according to the opcodes, and distributing each instruction to the corresponding execution unit.

Fig. 6(a) presents the proposed instructions for both CIM and SoftMax operations. In the function of CIM, various load and MAC operations involving different data dimensions and precisions can be illustrated within few concise and hardware-friendly CISC-like instructions of **AZI**, **WT**, **RD**, and **CIM**. In the function of SoftMax, specialized instructions for both top-*k* selection and SoftMax calculation are proposed, mainly include **LD**, **CMP**, **CLR**, and **SM** instructions. Therefore, only one instruction is needed for describing the whole selection cycle of one comparator, while only 10 instructions are needed in the 1024-to-64 selection task, compared to thousands of instructions if described in RISC-like style, as illustrated in Fig. 6(b). Detailed overhead comparison (i.e., time, area, energy) of instruction fetching and decoding between low-level RISC-like instructions and the proposed high-level CISC-like instructions based on the task of SoftMax calculation with the pre-processing procedure of 1024-to-64 selection is conducted, showing that these high-level instructions are much more friendly to the proposed hardware, since they can significantly reduce the instruction fetching and decoding overhead.

VI. Experimental Results

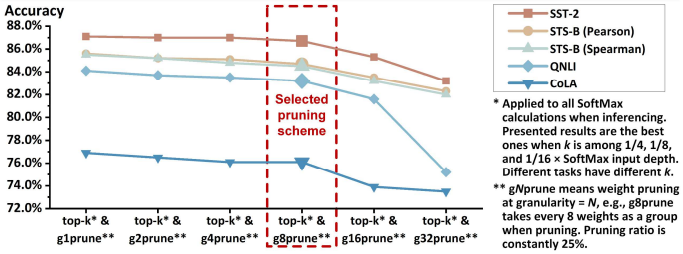
Algorithm Evaluation: To leverage the AZI and top-*k* selection features, two algorithmic techniques are adopted. First, weights are pruned with a granularity of N, where $N = 64$ / number of bits per weight, matching the pruning pattern with 64 DVSCs per row in the DCIM macro. Second, the SoftMax activation function with top-*k* selection is embedded into both training and inference stage. Consequently, for matrix multiplications (MM) involving a weight matrix, the row-wise sparsity comes from structured weight pruning strategy. While for MM of the SoftMax result with the Value matrix from attention layer, the row-wise sparsity comes from top-*k* selection, as the SoftMax outputs for non-top-*k* values are to '0'. Although these zeros are not intentionally grouped, the byte-wise sparsity, which can reach up to 93.75% in most aggressive top-*k* selection schemes, can translate to a row-wise sparsity of approximately $0.9375^8 \approx 0.60$.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:16 UTC from IEEE Xplore. Restrictions apply.

(a) Accuracy evaluation results

	SST-2	STS-B (Pearson)	STS-B (Spearman)	QNLI	CoLA
Baseline	92.8%	89.4%	89.0%	91.1%	82.4%
Top-k*	92.7%	88.9%	88.6%	90.5%	82.3%
Top-k* & g8prune**	86.7%	84.7%	84.5%	83.2%	76.1%

(b) Optimal pruning scheme exploration



* Applied to all SoftMax calculations when inferencing. Presented results are the best ones when k is among 1/4, 1/8, and 1/16 × SoftMax input depth. Different tasks have different k.

** gNprune means weight pruning at granularity = N, e.g., g8prune takes every 8 weights as a group when pruning. Pruning ratio is constantly 25%.

Fig. 7. (a) Accuracy evaluation results; (b) Optimal pruning scheme exploration.

We evaluate the model accuracy using the Q8BERT model on multiple GLUE tasks, with both inputs and weights in INT8 format. As shown in Fig. 7(a), the results reveal that weight pruning with a granularity of 8 is the most hardware-friendly configuration for our DCIM macro,. The combined impact of pruning and top-k selection introduces ~5.82% accuracy drop, which remains acceptable for natural language understanding task on lightweight edge devices. The accuracy drop solely due to top-k selection is less than 0.3%.

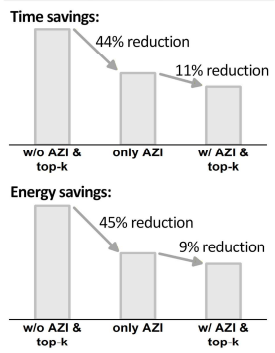
To validate the choice of 64 DVSCs per row, we analyzed the relationship between the pruning granularity and accuracy drop. Fig. 7(b) shows that the accuracy drops slightly when granularity changes from 1 to 8. However, a much steeper accuracy drop is observed for granularities of 16 and 32. This confirms that a granularity of 8, corresponding to 64 DVSCs per row, offers an optimal balance between hardware efficiency and algorithmic accuracy.

Hardware Measurement: Silicon measurements of the DCIM macro's write operations reveal that the energy cost of performing an AZI on one macro is 2.01~2.16× the energy required to write a single row, depending on the bit-flipping ratio. This indicates that when energy benefits of only write operations are considered, AZI can bring positive benefits when more than 3% rows are expected to store all-0 data and their write operations can be skipped. When the additional benefits during CIM operations are taken into account, the advantages of AZI become even more substantial.

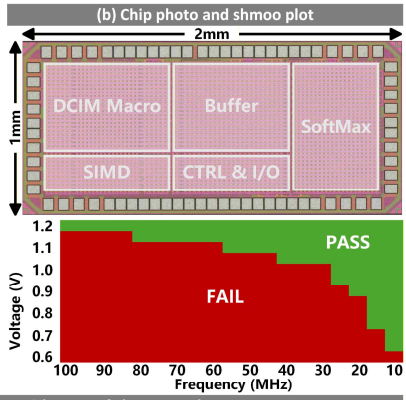
A detailed analysis of performance and efficiency gains from AZI and top-k selection is conducted and shown in Fig. 8(a). The AZI feature in DCIM macro reduces execution time and energy consumption by approximately 44% and 45%, respectively. The top-k selection strategy in the SMU provides further reduction of 11% in execution time and 9% in energy consumption. It should be noticed that although SoftMax calculation itself constitutes a relatively small portion of the total workload in Transformer inferencing tasks, its ability to generate high sparsity results significantly enhances the efficiency of subsequent MM when combined with AZI. This demonstrates that a synergistic effect between AZI and top-k selection, where each technique amplifies the benefits of the other, leading to highly efficient Transformer acceleration.

The TOKCIM is fabricated in 65nm CMOS process with a total area of 2mm^2, as shown in Fig. 8(b). It operates at 0.63-1.2V supply voltage under 10-100MHz clock frequency. When benchmarked on

(a) Time & Energy Savings by AZI & Top-k*1*2



(b) Chip photo and shmoo plot



(c) Comparison with state-of-the-art works

Reference	ISSCC'22 [3]	ISSCC'23 [1]	ASSCC'23 [4]	ESSERC'24 [2]	ISSCC'25 [5]	This Work
Technology (nm)	28	28	28	28	16	65
Supply Voltage (V)	0.6-1.0	0.64-1.03	0.6-1.0	0.85-1.0	0.45-0.85	0.63-1.2
Frequency (MHz)	80-240	20-320	80-240	50-285	60-450	10-100
Die Area (mm^2)	6.83	3.93	6.98	20.25	10.15	2.00
Peak Performance*1 (TOPS)	1.48	3.33*3	2.66*3	3.03*3	2.15	0.58*2*4
Energy Efficiency*1 (TOPS/W)	20.5	25.22*3	38.9*3	4.84*3	40.3 / 22.03*6	5.73*2*5 / 26.0*6
Area Efficiency*1 (TOPS/mm^2)	0.22	0.85*3	0.38*3	0.15*3	0.21 / 0.15*6	0.29*2*4 / 0.68*6
FoM*7 (TOPS/W · TOPS/mm^2)	4.51	21.44*3	14.78*3	0.73*3	3.30	17.68*2
Sparsity Handling	no	yes	no	yes	no	yes
SoftMax Speedup	no	no	no	no	no	yes

*1: Results are based on INT8 data format.
*2: Weight sparsity = 50% with pruning granularity = 8; top 1/16 inputs selected for SoftMax.
*3: Weight sparsity varies from 66% to 87.5%.
*4: Tested under 1.2V, 100MHz.
*5: Tested under 0.63V, 10MHz.
*6: For the sake of fairness, works other than 28nm are scaled to 28nm based on the scaling factor provided in [6].
*7: FoM is defined as "28-nm scaled energy efficiency × 28-nm scaled area efficiency".

Fig. 8. (a) Time and energy savings by AZI and top-k; (b) Chip photo and shmoo plot; (c) Comparison with SOTA works.

Q8BERT model and GLUE datasets with average weight pruning ratio of 0.5 and granularity of 8, TOKCIM achieves a peak energy efficiency of 5.73 TOPS/W (@0.63V, 10MHz), and a peak area efficiency of 0.29 TOPS/mm^2 (@ 1.2V, 100MHz). We compare our design with prior works [1]-[5] by normalizing all results to the 28nm node based on the scaling factor from [6]. As summarized in Fig. 8(c), TOKCIM achieves a competitive figure of merit (FoM), demonstrating state-of-the-art performance in both energy and area efficiency.

REFERENCES

[1] S. Liu *et al*., "A 28nm 53.8TOPS/W 8b Sparse Transformer Accelerator with In-Memory Butterfly Zero Skipper for Unstructured-Pruned NN and CIM-Based Local-Attention-Reusable Engine," *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2023.

[2] D. Kim *et al*., "DPIM: A 19.36 TOPS/W 2T1C eDRAM Transformer-in-Memory Chip with Sparsity-Aware Quantization and Heterogeneous Dense-Sparse Core," *2024 IEEE European Solid-State Electronics Research Conference (ESSERC)*, Bruges, Belgium, 2024.

[3] F. Tu *et al*., "A 28nm 15.59μJ/Token Full-Digital Bitline-Transpose CIM-Based Sparse Transformer Accelerator with Pipeline/Parallel Reconfigurable Modes," *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2022.

[4] R. Guo *et al*., "CIMFormer: A 38.9TOPS/W-8b Systolic CIM-Array Based Transformer Processor with Token-Slimmed Attention Reformulating and Principal Possibility Gathering," *2023 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, Haikou, China, 2023.

[5] S. Moon *et al*., "T-REX: A 68-to-567μs/Token 0.41-to-3.95μJ/Token Transformer Accelerator with Reduced External Memory Access and Enhanced Hardware Utilization in 16nm FinFET," *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, San Francisco, CA, USA, 2025.

[6] W. Huang, K. Rajamani, M. R. Stan and K. Skadron, "Scaling with Design Constraints: Predicting the Future of Big Chips," in *IEEE Micro*, vol. 31, no. 4, pp. 16-29, July-Aug. 2011, doi: 10.1109/MM.2011.4

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:16 UTC from IEEE Xplore. Restrictions apply.